

機器人強化學習實作課

從模仿學習到連續動作控制

7 種演算法

BC → Q-Learning → DQN → DDPG → TD3 → SAC → PPO

課程敘事主線



每個方法都在**回答前一個方法留下的問題**，形成一條完整因果鏈。

幕	主題	方法
第一幕	為什麼需要 RL ？	BC / Q-Learning / DQN
第二幕	連續動作控制	DDPG / TD3 / SAC
第三幕	穩定訓練與部署	PPO

第一幕

為什麼需要強化學習？

- 方法 1：Behavioral Cloning — 模仿看起來很直觀...
- 方法 2：Q-Learning — RL 的基礎語言
- 方法 3：DQN — 神經網路解決無限狀態

方法 1 | Behavioral Cloning

最直觀的想法：讓機器人看人怎麼做，照著學

- 收集人類（或現有 policy）的示範軌跡
- 直接做**監督學習**：讓 policy 模仿專家動作

```
# 訓練
loss = MSE(policy(state), expert_action)

# 推論
action = policy(state)
```

優點：簡單、不需要設計獎勵函式、不需要探索

問題是什麼？→ 見下一張

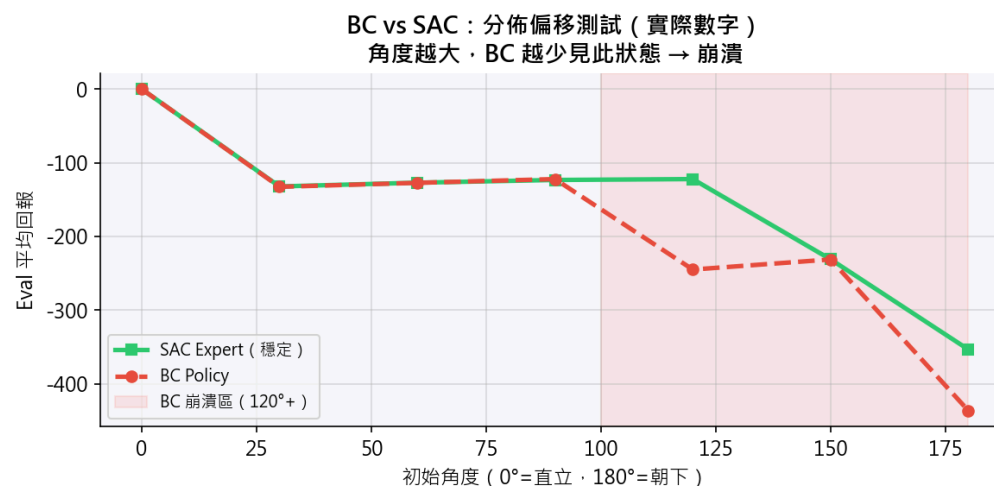
Distribution Shift (分佈偏移)

訓練時：只看到專家軌跡附近的狀態（擺錘接近直立）

測試時：稍微偏離 → policy 不知道怎麼辦 → 錯誤累積 → 崩潰

初始角度	SAC Expert	BC Policy
0° – 90°	-123	-123 ✓
120°	-122	-245 ✗
180°	-354	-437 ✗

BC 不是沒用，在見過的狀態附近它模仿得很好
問題是：真實機器人碰到略微不同的初始狀態 → 累積誤差 → 崩潰



為什麼需要 RL ?

BC 的根本問題：

訓練時只見好的狀態

- 測試時若偏離 → 不知所措
- 錯誤累積 (Compounding Error)
- 策略崩潰

我們需要讓 agent：

1. 自己探索各種狀態（包括糟糕的狀態）
2. 從錯誤中修正，不只複製專家

→ 這就是強化學習 (Reinforcement Learning)

方法 2 | Q-Learning

RL 的基礎語言：給機器人一個獎懲訊號，讓它自己學

三個核心概念

① $Q(s, a)$ ：在狀態 s 採取動作 a 的長期期望回報

② Bellman Equation (TD 更新)：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \cdot \max_{a'} Q(s', a')}_{\text{TD 目標}} - \underbrace{Q(s, a)}_{\text{TD 誤差}} \right]$$

③ ϵ -greedy 探索：

- 以 ϵ 機率隨機探索（避免陷入區域性最優）
- 以 $1-\epsilon$ 機率選最大 Q 值的動作（利用已有知識）

Q-table 的致命問題

FrozenLake (4×4 格子) :

- 16 個格子 × 4 個動作 = **64 個 Q 值**
- Q-table 完全可行 ✓

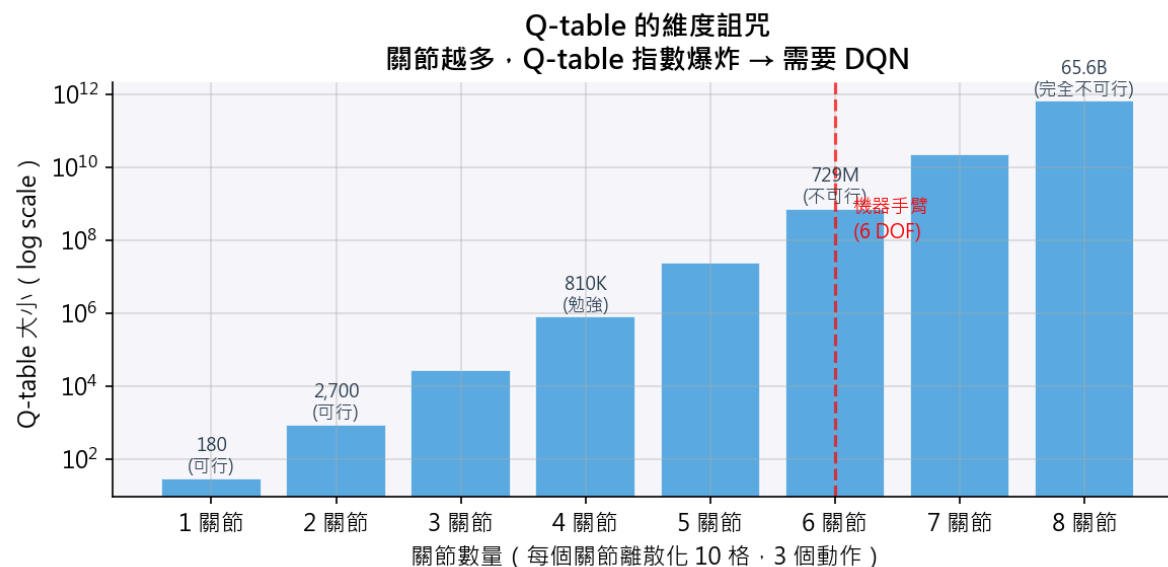
CartPole (4 個連續變數) :

- 每個維度離散化 10 格 → $10^4 \times 2 = 20,000$ Q 值
- 勉強可行...

機器手臂 (6 個關節) :

- $10^6 \times 3^6 = 729,000,000$ Q 值
- 記憶體放不下 ✗
- 學習時間無限長 ✗

結論：Q-table 解決不了真實機器人的連續狀態問題



方法 3 | DQN (Deep Q-Network)

用神經網路取代 Q-table，解決無限狀態問題

DQN 的兩個關鍵設計

① Experience Replay (經驗回放)

```
# 不直接學當前轉移，而是從 Buffer 隨機取樣  
batch = replay_buffer.sample(batch_size)  
# → 打破時序相關性，穩定訓練
```

② Target Network (目標網路)

```
# TD 目標由慢更新的 target_net 決定  
y = r +  $\gamma$  · target_net(s').max()  
# → 固定學習目標，防止「追著自己跑」
```

DQN 訓練結果

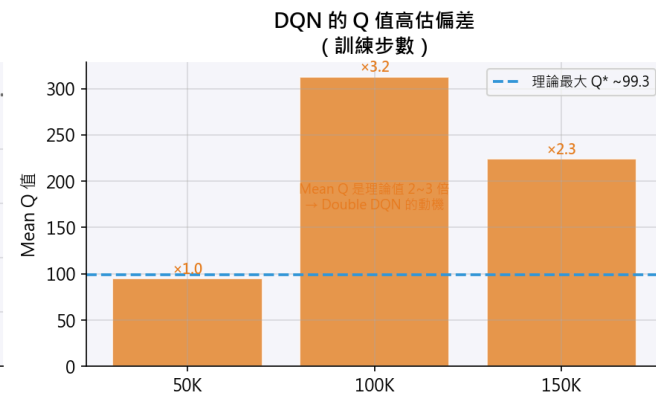
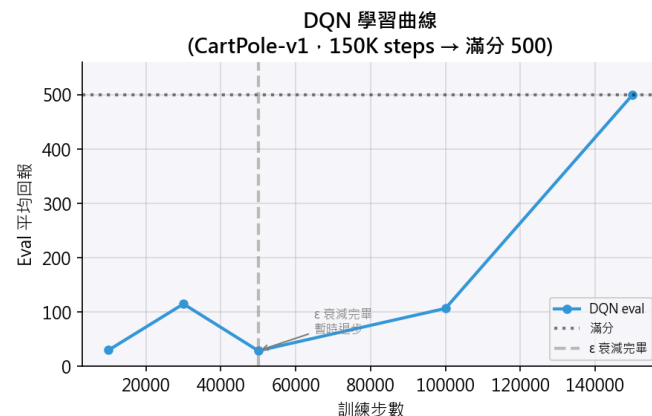
環境：CartPole-v1

結果：

- 150K 步：eval = 500 (滿分) 🎉
- Q 值高估：mean Q 高達 312 (理論值 ~50)
- 高估來自 `max` 運算元的系統性偏差

DQN 的限制：

- 輸出是離散動作 (左 / 右)
- 無法直接控制馬達的連續角度
- 機器手臂需要扭矩值，不是「往左或往右」



第二幕

連續動作——進入真實機器人領域

- 方法 4：DDPG — Actor-Critic，直接輸出連續扭矩
- 方法 5：TD3 — 修掉 DDPG 的三個已知缺陷
- 方法 6：SAC — 自動平衡探索與利用

方法 4 | DDPG (Deep Deterministic Policy Gradient)

如何控制連續的馬達扭矩？

Actor-Critic 架構

```
Actor 網路：state → action (連續值，扭矩  $\in [-2, 2]$ )  
Critic 網路：(state, action) → Q 值 (這個動作好不好？)
```

與機器手臂的對應：

- Actor = 關節角度控制器 (「現在這個姿態，施加多少扭矩？」)
- Critic = 評估這個姿態的長期價值

核心程式碼：

```
action = actor(state)          # 連續動作  
actor_loss = -critic(state, actor(state)).mean()  # 讓 Q 值更高
```

DDPG 訓練結果

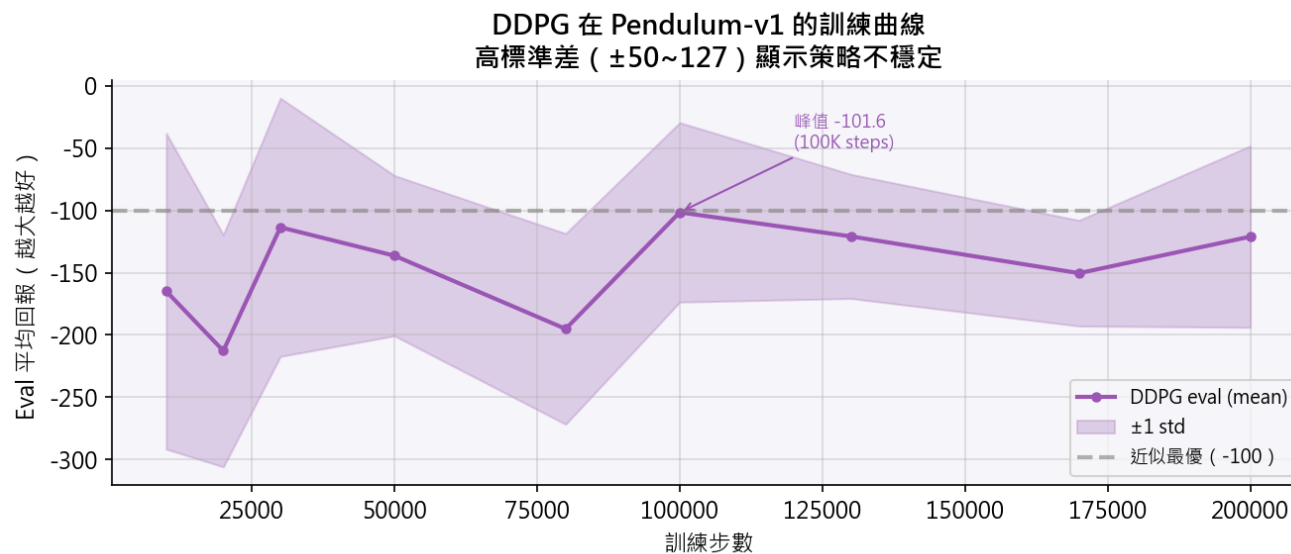
環境：Pendulum-v1（單關節扭矩控制）

結果：

- 最佳 eval = **-101.6**（接近最優 -100）🎉
- 但訓練曲線震盪劇烈
- 高標準差：策略不穩定

DDPG 的三個已知問題：

1. Q 值高估（max 運算元偏差）
2. Actor **更新太快**，Critic 還沒學好
3. 過擬合特定動作，**探索不足**



方法 5 | TD3 (Twin Delayed Deep Deterministic)

DDPG 的三個問題，TD3 一次修掉

問題	TD3 的解法	程式碼改動
Q 值高估	Twin Critics ：取兩個 Critic 的最小值	+1 個 Critic 網路
Actor 更新太快	Delayed Actor Update ：每 2 步才更新 Actor	<code>if step % 2 == 0</code>
探索不足	Target Policy Smoothing ：目標動作加入雜訊	<code>a' += clip(noise, -c, c)</code>

完整 TD 目標：

```
a_next = target_actor(s') + clip(noise, -c, c) # 平滑雜訊
q1, q2 = target_critic1(s', a_next), target_critic2(s', a_next)
y       = r +  $\gamma$  * min(q1, q2)                # Twin Critic 取最小
```

TD3 vs DDPG 對比

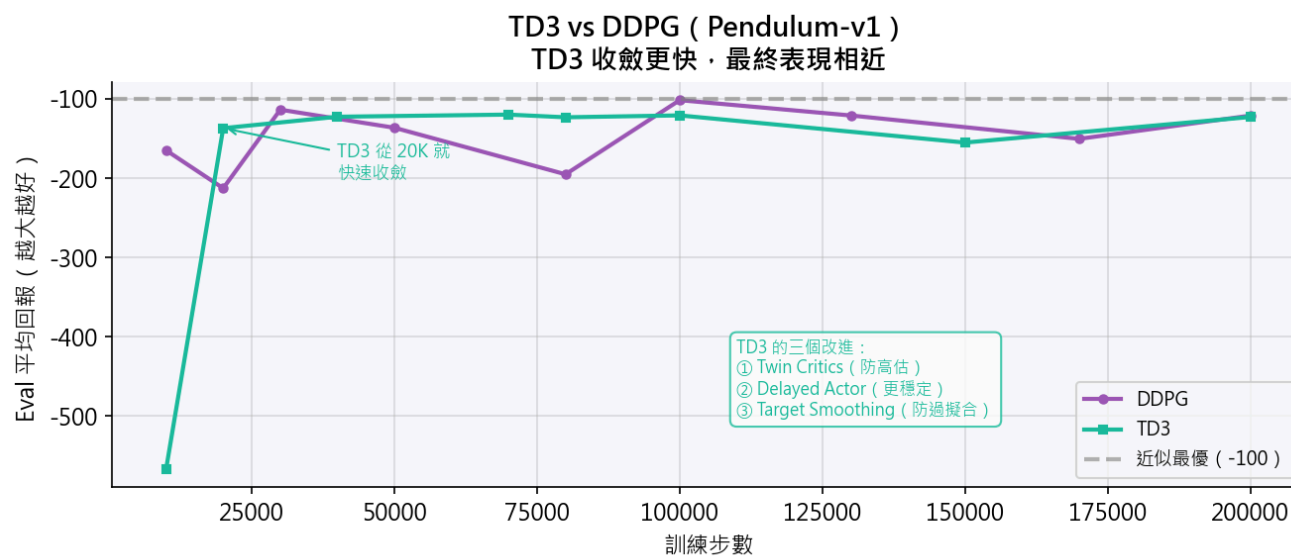
環境：Pendulum-v1

演算法	最佳 Eval	收斂步數
DDPG	-101.6	100K
TD3	-119.8	70K

- TD3 峰值略低，但**收斂更快、更穩定**
- 標準差更小：部署時更可靠

TD3 的限制：

- 探索仍依賴**人工加雜訊**（固定探索強度）
- 多關節任務容易陷入**區域性最優**
- 如何讓機器人自動決定探索多少？



引入資訊熵，讓機器人自動平衡探索與利用

一般 RL：最大化 $\sum r_t$
SAC： 最大化 $\sum [r_t + \alpha \cdot H(\pi(\cdot|s_t))]$
↑ 鼓勵高 Entropy (多樣性)

- 高 Entropy = 保持探索 = 不陷入區域性最優
- 不只學「一種解法」，而是「任何合理姿態都行」
- 部署到真實硬體時更穩健

16

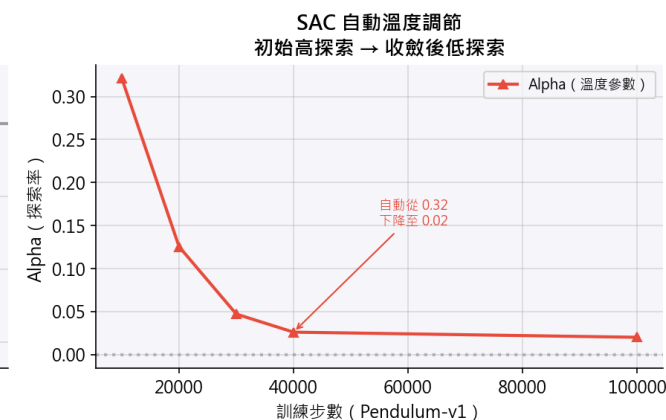
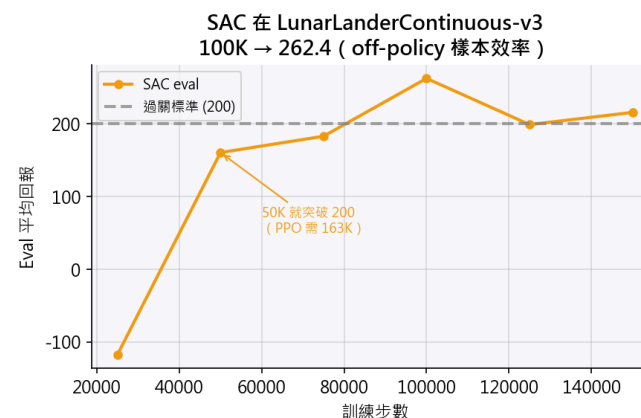
SAC 自動溫度調節

- 訓練初期： α 高（強迫探索）
- 策略收斂後： α 自動下降（停止過度探索）
- 完全自動，無需手動調

SAC 在不同環境的表現：

環境	Eval	步數
Pendulum	-171.8	100K
LunarLanderCont.	262.4	100K

Pendulum 太簡單，SAC 的優勢在複雜多軸任務才顯現



三種連續控制演算法對比

Pendulum-v1（回報越靠近 0 越好）：

演算法	最佳 Eval	步數	訓練穩定性
DDPG	-101.6	100K	低（高方差）
TD3	-119.8	70K	中（更穩）
SAC	-171.8	100K	高（但過度探索）

LunarLanderContinuous-v3：

演算法	eval ≥ 200 達成步數
PPO	~163K 步
SAC	~50K 步 ← 樣本效率碾壓

SAC 的競爭力不在極簡任務，而在多軸連續控制

為什麼 SAC 是業界標準

真實機器人部署的三個需求：

需求	SAC 的表現
樣本效率	✓ Off-policy，可重用舊資料
訓練穩定	✓ Entropy 正則化防止策略崩潰
超引數敏感度	✓ α 自動調整，減少調參負擔

SAC 的限制：

- 計算成本略高（兩個 Critic + 一個 Actor + Value Net）
- 在模擬環境快速迭代時，on-policy 的 PPO 更常用

→ 第三幕：為什麼模擬環境的 SOTA 是 PPO？

第三幕

穩定訓練，走向實際部署

- 方法 7：PPO — 為什麼 OpenAI、DeepMind 的機器人論文幾乎都用 PPO？

方法 7 | PPO (Proximal Policy Optimization)

先看 REINFORCE 的問題

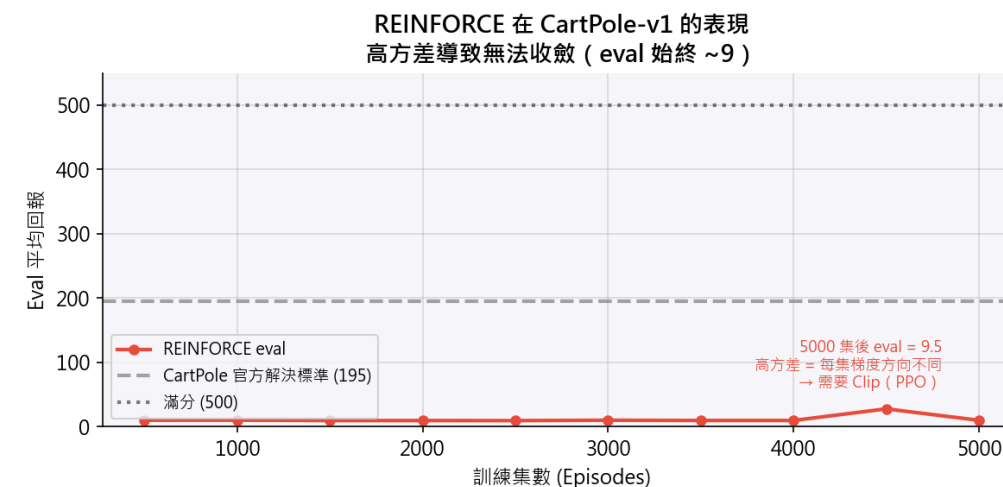
REINFORCE (最基本的 Policy Gradient) :

```
loss = -log  $\pi(a|s)$   $\times$  G_t # 直接對回報加權做梯度上升
```

問題：每次更新幅度無限制

- 一次更新走太遠 $\rightarrow$ 策略崩潰
- 崩潰後很難恢復

實際結果 (我們跑的) :



PPO 的核心創新：Clipped Objective

PPO 的限制更新幅度：

```
ratio =  $\pi_{\text{new}}(a|s)$  /  $\pi_{\text{old}}(a|s)$     # 新舊策略的比值

loss = -min(
    ratio * A,                # 原始目標
    clip(ratio, 1- $\epsilon$ , 1+ $\epsilon$ ) * A    # 裁剪版本
)
# 「每次更新不能走太遠，
# 新策略和舊策略的差距要在安全範圍內。」
```

優點：

- 單次更新有上界 → 策略不會崩潰
- 可以在同一批資料上多次更新（`n_epochs`）
- 實作簡單，穩定性極高

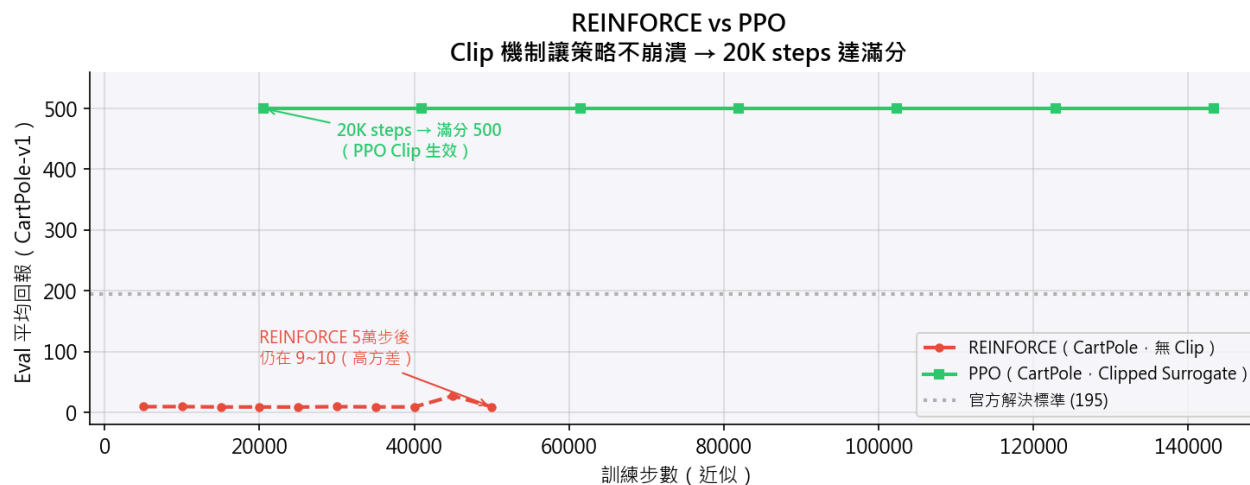
REINFORCE vs PPO

CartPole-v1 :

演算法	最終 Eval
REINFORCE	9.5 (卡住)
PPO	500 (滿分)

LunarLander-v3 :

- PPO eval $\approx$ 284
- 穩定收斂，訓練曲線平滑



SAC vs PPO 使用場景

	SAC	PPO
訓練方式	Off-policy（重用舊資料）	On-policy（每次更新後丟棄）
樣本效率	✓ 高	低
訓練穩定性	高	✓ 很高
計算需求	中	低
適用場景	真實機器人部署	模擬環境快速迭代

實際選擇：

- 模擬器便宜（可以跑百萬步）→ PPO
- 真實機器人資料貴（每步都要錢）→ SAC

課程總結

7 種演算法的完整因果鏈

完整因果鏈回顧

方法	核心問題	解法	環境	結果
BC	有示範就夠了嗎？	監督學習模仿	Pendulum	-145（分佈內）
Q-Learning	RL 的基礎語言？	Bellman + ϵ -greedy	FrozenLake	收斂 ✓
DQN	無限狀態怎麼辦？	神經網路 + Replay + Target	CartPole	500 ✓
DDPG	連續動作怎麼辦？	Actor-Critic	Pendulum	-101.6 ✓
TD3	DDPG 為何不穩？	Twin Q + Delay + Noise	Pendulum	-119.8 ✓
SAC	如何自動探索？	最大熵 + 自動 α	Pendulum+LunarLander	-172 / 262 ✓
PPO	如何穩定更新？	Clip + On-policy	CartPole+LunarLander	500 / 284 ✓

下一步：走向真實機器人

課後延伸方向：

主題	方法	解決的問題
稀疏獎勵	HER (Hindsight Experience Replay)	「幾乎從不成功」的任務
更好的模仿	GAIL	BC + RL 的結合
減少真實資料需求	Model-Based RL (Dreamer)	用世界模型想像未來
模擬→真實	Sim-to-Real Transfer	Domain Randomization

核心訊息：

機器人 RL 的每一步進展，都是在解答前一個方法的限制。
這條因果鏈還在繼續——下一個突破就在你的研究中。

謝謝

問題討論